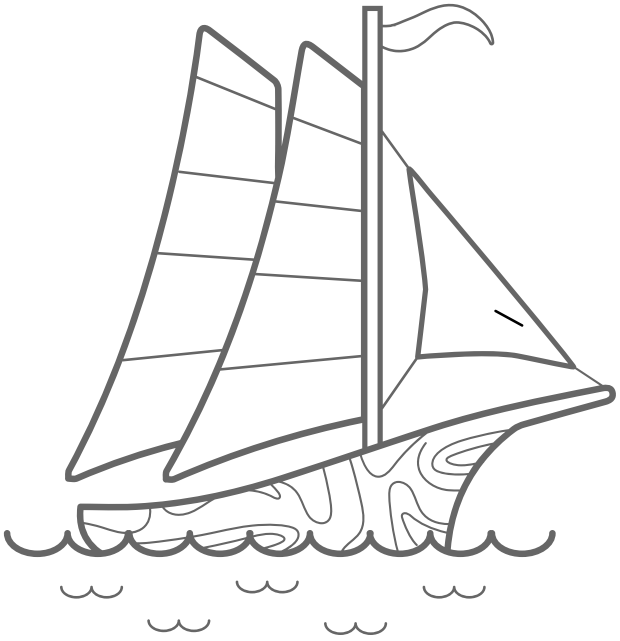
Hes·so VALAIS WALLIS
School of Engineering

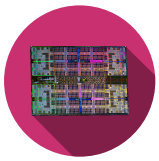
LangQuest

User Guide



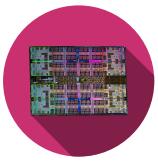
Computer Architecture
Information and Communication Technology •
Computer Architecture • Silvan Zahno

Silvan Zahno, **Axel Amand**
09.09.2026 • v1.0.0 • final



Contents

1 Introduction 3
2 Installation 4
2.1 Rust Toolchain 4
2.1.1 Installation 4
2.1.2 Install `cargo` addons 5
2.1.3 Verification 6
2.2 LangQuest 7
2.2.1 Installation 7
2.2.2 Quick use 8
2.2.3 Mapping files to editor 8
2.3 Zed Code Editor 8
2.3.1 Installation 8
2.3.2 Verification 9
2.4 Just - Command Runner 9
2.4.1 Installation 9
2.4.2 Verification 9
2.5 GH CLI 10
2.5.1 Installation 10
2.5.2 Verification 11
3 LangQuest Guide 12
3.1 Run LangQuest 12
3.2 Overview page 13
3.2.1 Shortcuts summary 14
3.3 Exercises page 15
3.3.1 Theory 15
3.3.2 Tasks 15
3.3.3 Debug 15
3.3.4 Output 16
3.3.5 Tips and Solution 16
3.3.6 Shortcuts summary 17
3.4 Troubleshooting 18
Glossary 19



1 Introduction

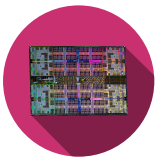
LangQuest as presented by its GitHub page:

> 
>
>
> A terminal-based, interactive programming exercise runner. Inspired by Rustlings and 100 Exercises to Learn Rust, LangQuest (lq) extends the concept to multiple languages - work through hands-on exercises in Rust, Go, C++, Python, RISC-V Instruction Set Architecture (RISC-V) assembly, and Markdown with real-time feedback, progress tracking, and a built-in hint system.

This tool is developed by Zahno Silvan and released under the MIT License (MIT).
The source code can be found under https://github.com/tschinz/langquest.

LangQuest is an interactive tool whose interface runs in the command line thanks to Ratatui and allows you to work on hands-on exercises in Rust, Go, C++, Python, RISC-V assembly, and Markdown with real-time feedback, progress tracking, and a built-in hint system.

The following document provides a comprehensive user guide for LangQuest, covering installation, usage, and troubleshooting to help you get the most out of this interactive learning tool.



2 Installation

This section covers the installation of all tools related to LangQuest. Work through the subsections in order - some tools depend on others being installed first.

2.1 Rust Toolchain

Figure 1 - Rust Programming Language

Various tools are installed through Cargo, the Rust package manager.

Rust is installed and managed through **`rustup`** - the official Rust toolchain installer. **`rustup`** handles installation of the compiler (**`rustc`**), the package manager and build tool (**`cargo`**), and additional targets or components.

2.1.1 Installation

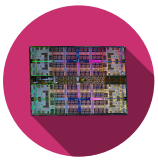
Visit rust-lang.org/tools/install for full instructions.

Open a terminal and run:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Follow the on-screen prompts (the default installation is recommended). After installation, re-open your terminal or reload the environment:

```
source "$HOME/.cargo/env"
```



Download and run **`rustup-init.exe`** from https://win.rustup.rs/.

If Microsoft Visual C++ (MSVC) tools are not installed, choose to install them when prompted:

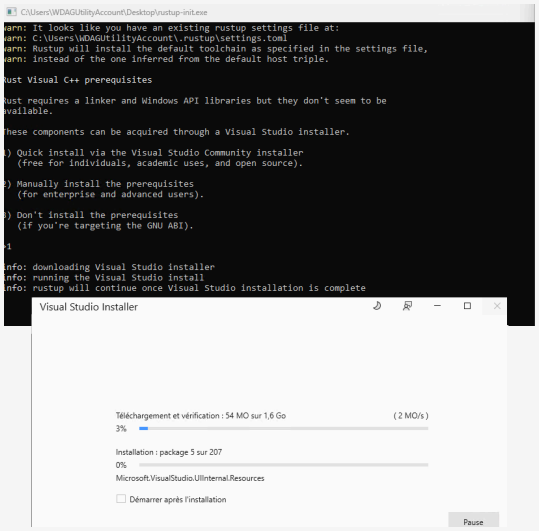


Figure 2 - MSVC installation through rustup

Then proceed with installation (the standard installation is recommended):

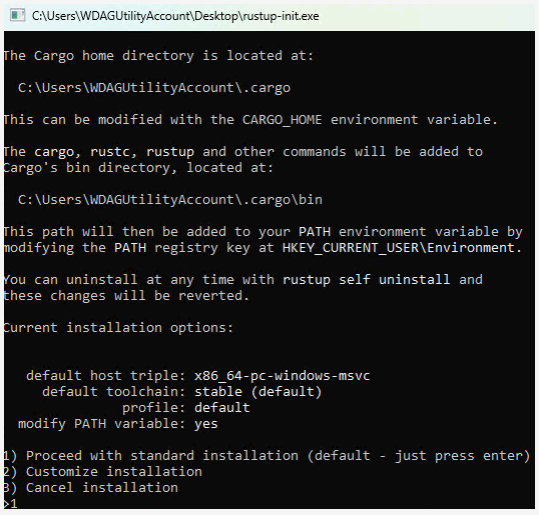
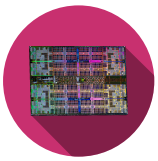


Figure 3 - Rust installation through rustup on Windows

2.1.2 Install `cargo` addons

In a new terminal, run the following commands to install useful **`cargo`** addons for code formatting and linting:

```
# Install rustfmt for code formatting
rustup component add rustfmt
# Install clippy for linting and code analysis
rustup component add clippy
```

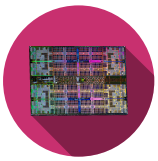


2.1.3 Verification

Verify that the following command prints a version number without error:

```
rustup --version
rustc --version
cargo --version
cargo fmt --version
cargo clippy --version
```

All commands should print a version number. If they fail, ensure **`$HOME/.cargo/bin`** (macOS/Linux) or **`%USERPROFILE%\.cargo\bin`** (Windows) is in your **`PATH`**.



2.2 LangQuest

Figure 4 - LangQuest - vocabulary quiz tool

LangQuest is a terminal-based vocabulary quiz tool used to practice programming terminology and concepts. It is distributed as binary under LangQuest on Github.

2.2.1 Installation

```
curl -fsSL https://raw.githubusercontent.com/tschinz/langquest/
refs/heads/main/scripts/install_latest_release.sh | sh
```

This installs LangQuest under **$HOME/.local/bin**.

```
irm https://raw.githubusercontent.com/tschinz/langquest/refs/heads/
main/scripts/install_latest_release.ps1 | iex
```

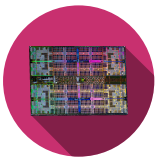
This installs LangQuest under **%LOCALAPPDATA%\Programs\lq\bin\lq.exe**.

Verify that the following command prints a version number without error:

```
lq -k
```

Check that the keys correspond to the followings:

```
progress_key_sha256:
869e730b3d6be463c5474a41f802711943e330827a7c4ddd6bcdf2dc9f4abc3a
attest_key_sha256:
af30b6a2512c11e8015da1522b55d61926f96591c3193a63f2d1b64f6f53b5b7
solution_key_sha256:
1d142c246028f02e928682c88b3280412d4ff82b0cbc275db68a236947c89283
```



2.2.2 Quick use

To run it, head to a folder containing LangQuest exercises and run it:

```
cd <path/to/exercises>
lq
```

The program will automatically detect the exercises layout and content.

If the command is run for the first time in a folder, it will create multiple configuration files:

- **`lq.toml`** - configuration file for LangQuest
  - Should be committed to version control
- **`.lq.progress`** - progress file to track your progress (encrypted, do not edit)
  - Should be committed to version control
- **`.lq.attest`** - attestation file for offline usage (encrypted, do not edit)
  - Machine specific, **do not commit** to version control

2.2.3 Mapping files to editor

You can define the editor for LangQuest to open exericse files. This config resides in **`lq.toml`**. By default it is configures to use **`zed`** (https://zed.dev/). You can change it to your preferred editor, e.g. **`code`** for VSCode or **`subl`** for Sublime Text.

```toml
[ide]
bin = "/usr/local/bin/zed"
cmd = "<ide> <file>"
```

2.3 Zed Code Editor

Figure 5 - Zed - high-performance code editor

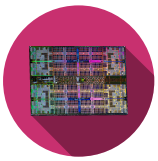
Zed is a high-performance, multiplayer code editor built in Rust. It is the primary editor used throughout this guide.

2.3.1 Installation

Visit zed.dev and download the installer for your platform.

Download the **`.dmg`**, open it and drag *Zed* to the *Applications* folder.

Install the Zed CLI tool by running the following command in Zed Press **`cmd-shift-p`** and type **`cli:install cli binary`**



Open a terminal and run the official installer script:

```
curl https://zed.dev/install.sh | sh
```

Download and run the **.exe** installer.
Add **zed.exe** to your **PATH** if the installer does not do it automatically.

2.3.2 Verification

Open Zed and confirm the welcome screen appears. Sign in with your GitHub account to enable Artificial Intelligence (AI) features.

Execute the following command in a terminal to verify that Zed is installed and available in your **PATH**:

```
zed --version
```

2.4 Just - Command Runner

just is a handy command runner (similar to **make**) that reads a **justfile** in your project. It is distributed as a binary Rust crate and installed via **cargo**.

2.4.1 Installation

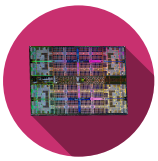
```
cargo install just
```

This may take a few minutes on the first install as it downloads and compiles the crate and its dependencies.

2.4.2 Verification

Verify that the following command prints a version number without error.

```
just --version
```



2.5 GH CLI

GitHub CLI is a command-line tool for interacting with GitHub repositories and services. It is used to manage pull requests, issues, and other GitHub features directly from the terminal.

It is used by LangQuest to identify the files based on the registered Github user.

2.5.1 Installation

Use brew to install:

```
brew install gh
```

Install it from your package manager if available, e.g.: Debian/Ubuntu

```
sudo apt install gh # Ubunutu
sudo dnf install gh # Fedora
```

Either use WinGet if installed on your system:

```
winget install --id GitHub.cli
```

Or download the installer for your system directly from cli.github.com and run it.

Then run the following command once to authenticate with your GitHub account. Follow the prompts to complete the authentication process by signing in via a web browser or using a personal access token:

```
gh auth login
```

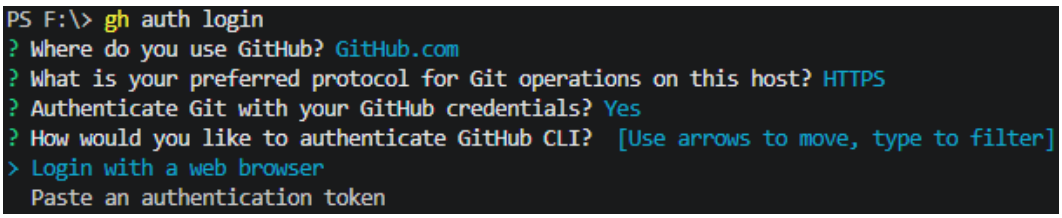
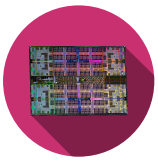


Figure 6 - GitHub CLI installation



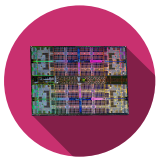
2.5.2 Verification

Run

```
1 gh auth status
```

to ensure the account is correctly linked:

```
PS F:\dev_work\langquest> gh auth status
github.com
  ✓ Logged in to github.com account Qarmma (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_************************************
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
```



3 LangQuest Guide

3.1 Run LangQuest

LangQuest does not require any specific location for the exercises to be put in, but requires the exercises tree to follow a specific layout:

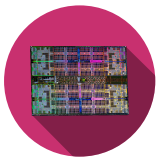
```
my-exercises/
├── lq.toml                     <-- auto-created by lq on first run
├── 01-basics/                  <-- module (prefixed with NN-)
│   ├── 01-hello-world/         <-- exercise (prefixed with NN-)
│   │   ├── 01-theory.md
│   │   ├── 02-task.md
│   │   ├── main.rs
│   │   └── solution/
│   │       ├── main.rs
│   │       └── solution.md
│   └── 02-variables/
│       └── ...
└── 02-control-flow/
    └── ...
```

Save the exercises for your current course somewhere under a dedicated folder.

In a terminal, navigate to the folder containing the exercises and run **`lq`** to start LangQuest:

```
cd <path/to/exercises>
lq
```

It is recommended to run it in a terminal opened in Zed to directly open the exercise files in the same editor. The following guide's screenshots are from Zed.



3.2 Overview page

Open the folder containing the exercises in Zed:

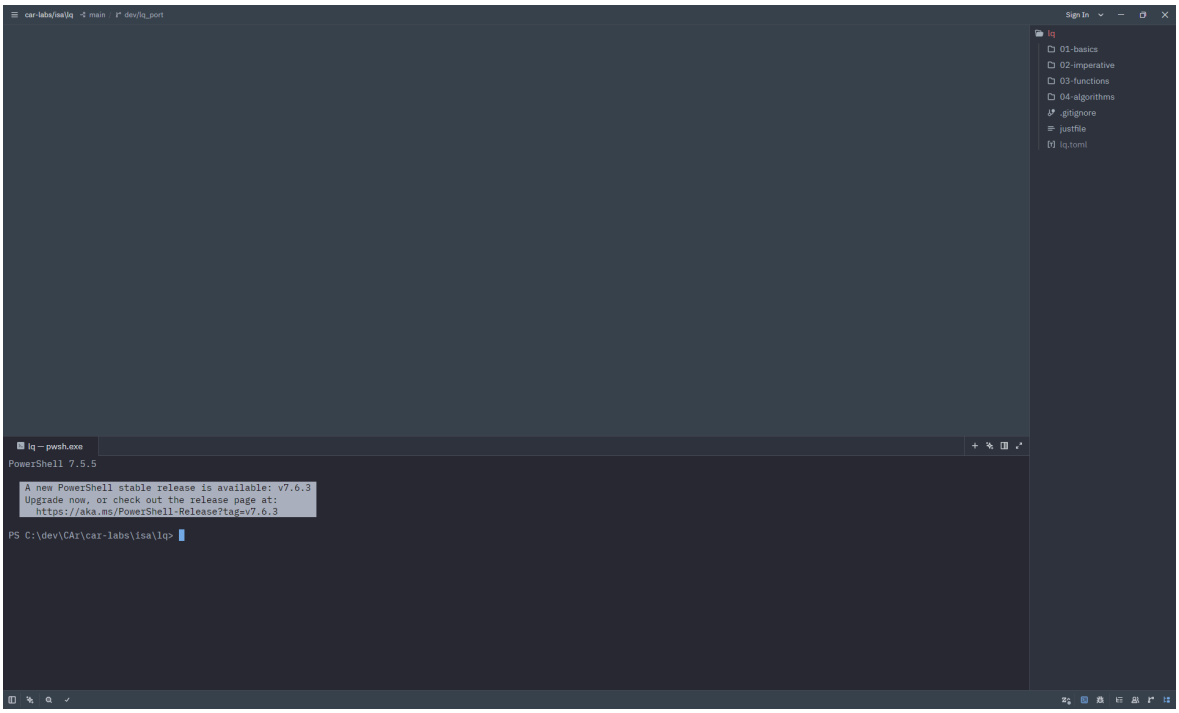


Figure 7 - Open exercises in Zed

Open the terminal through **`View`** ⇨ **`Terminal Panel`**, type **`lq`** then press **`Enter`**:

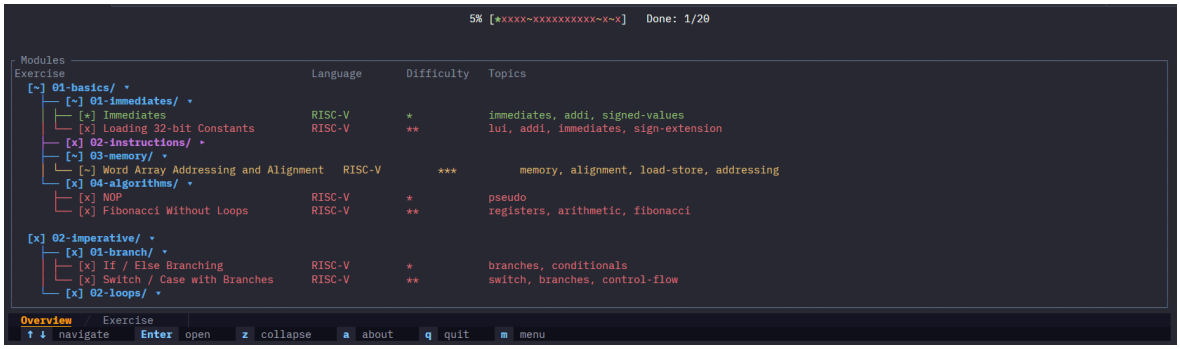
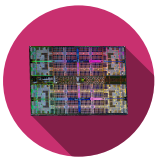


Figure 8 - **`lq`** in Zed

> If the program does not launch, read the terminal. Most errors come from running **`lq`** outside an exercises folder, or from an invalid exercise layout.



The program interface is as follows:

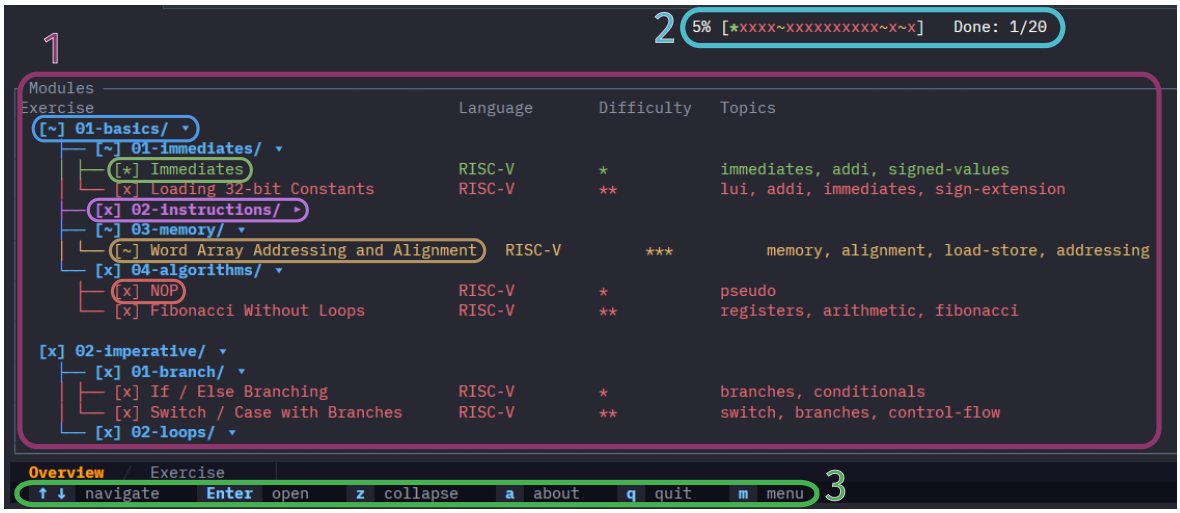


Figure 9 - **lq** overview page

1. Exercises list

Found exercises are shown in an ordered fashion. Iterate through them with the arrow keys ↓ ↑ and select one with **Enter**.

- The blue lines indicate modules/groups of exercises. Click on ***z*** to collapse/expand the modules.
- The purple line represent the cursor position.
- The green line indicates fully completed exercises.
- The yellow line indicates partially completed exercises.
- The red line indicates exercises that have not been completed.

For each exercise, the view shows the language (Rust, Go, C++, Python, RISC-V assembly, and Markdown), its difficulty level compared to relative exercises in the same module, and the topics it covers.

2. Progression

The progress bar shows the number of exercises completed in the course. It is updated to follow the completion of each exercise by showing:

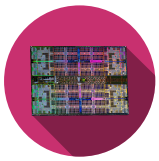
- *: fully completed exercise
- ~: partially meeting the requirements for the exercise
- x: not meeting the requirements

3. Menu

List of the keyboard strokes available in the program for the currently shown menu. Click on ***m*** to hide/show the menu.

3.2.1 Shortcuts summary

- ↓ ↑: navigate through the exercises list
- **Enter**: expand module / select an exercise
- ***z***: collapse/expand all modules in the list
- ***a***: show the about page
- ***q*** / ***Esc***: quit the program
- ***m***: show/hide the menu with the shortcuts for the current view



3.3 Exercises page

The views are walked through by using the ← → keys. Theory ⇔ Task ⇔ Debug ⇔ Output ⇔ Solution (if unlocked).

3.3.1 Theory

Upon entering an exercise, the theory tab is shown. It contains a short description and reminders to the core concepts needed to solve the exercise.

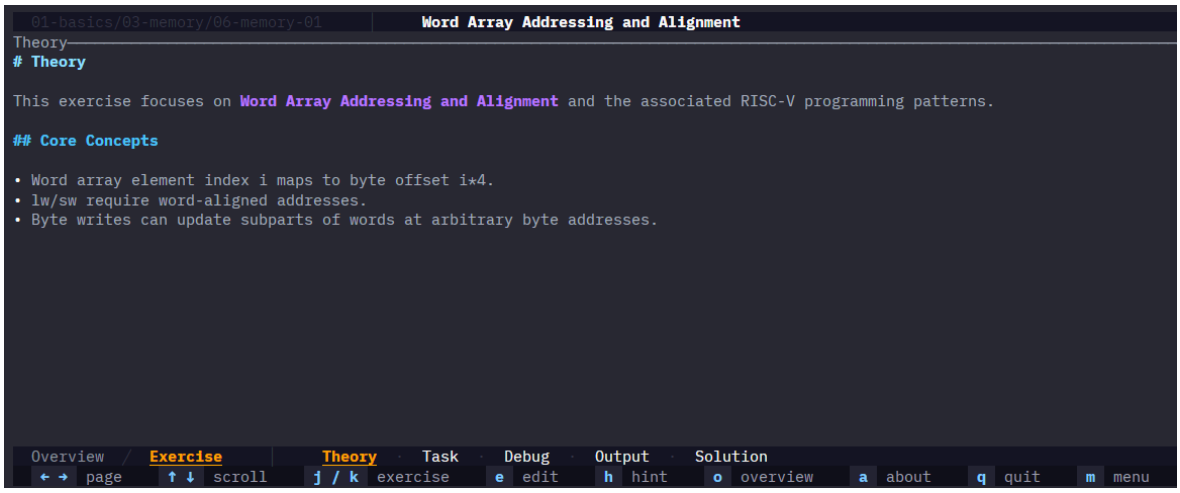


Figure 10 - Theory tab

3.3.2 Tasks

The tasks view contains the list of tasks to be completed for the exercise, code snippets, guidelines …

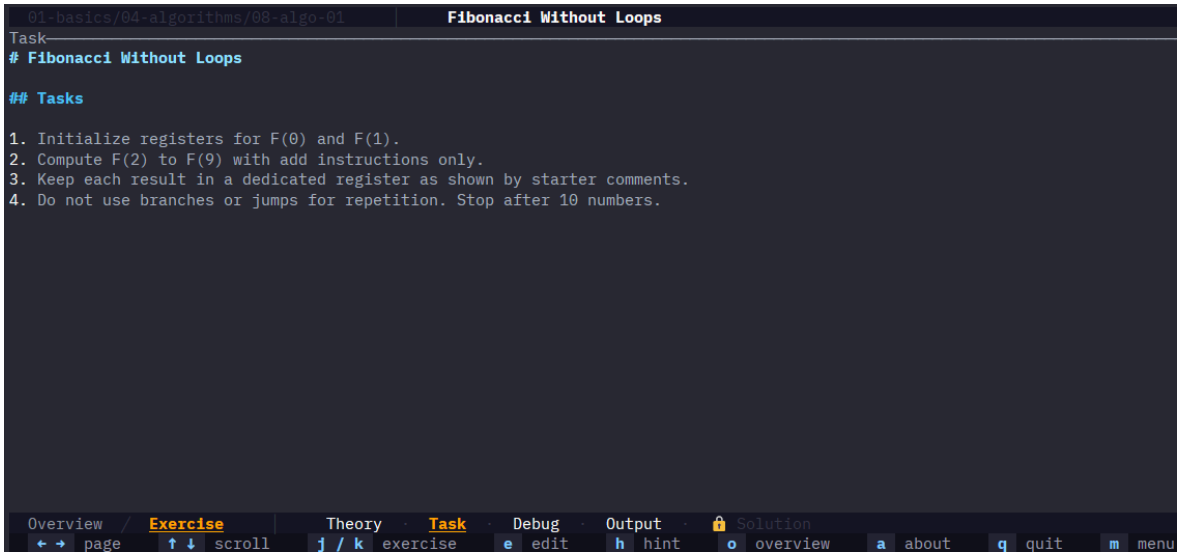


Figure 11 - Tasks tab

3.3.3 Debug

The debug window is used to show the output of **`stdout`** and **`stderr`** of the program being tested. *This window is not useful for **`RISC-V`** and **`Markdown`**.*

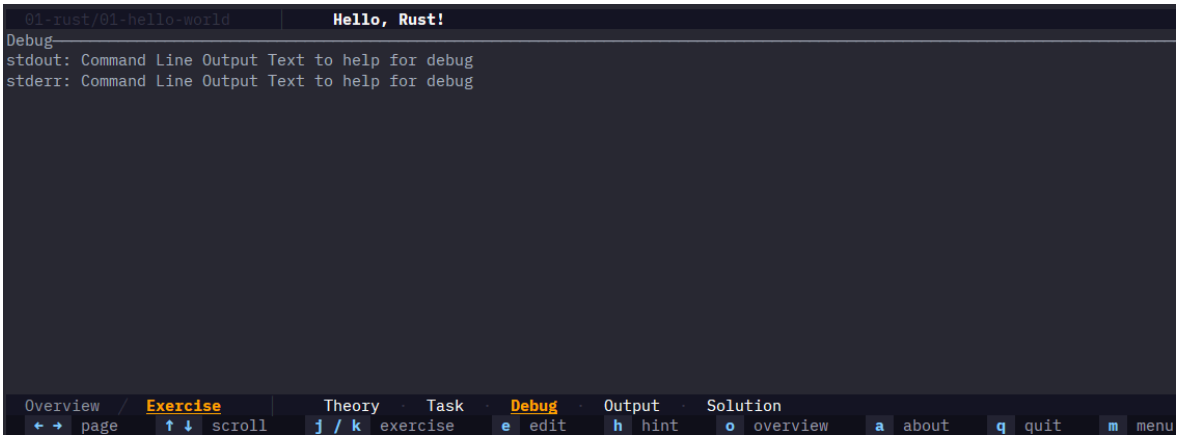


Figure 12 - Debug tab

3.3.4 Output

The output window is used to show the result of tests. The tests are defined in the exercise code directly, so refer to those to understand them better in case of failures.
It shows the total of tests passing and which ones are failing, the format being language specific.
The threshold on the right is the minimum percentage of tests that must pass for the exercise to be considered partially completed or fully completed.
This threshold is mostly targeted for Markdown exercises due to how tests are performed on them.

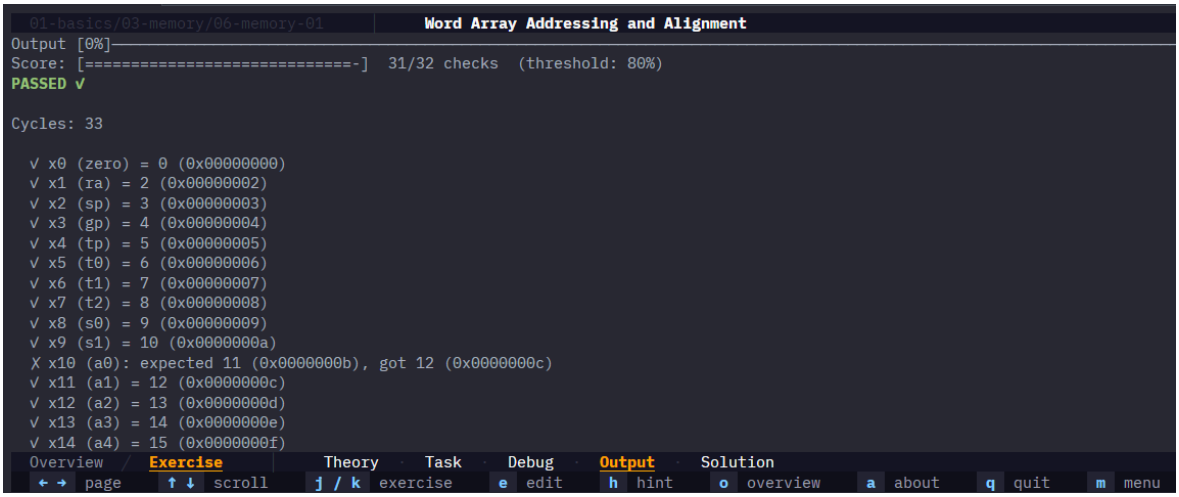


Figure 13 - Output tab

Press ***e*** to open the exercise in the default editor set before in Section 2.2.3. If mapped with the same editor used for running **`lq`** (e.g. Zed), it opens in a new tab in the same window.

3.3.5 Tips and Solution

By default, the solution is locked as shown by the lock icon 

It is unlocked under one of the two conditions:

- The threshold completion for the exercise is met.
- All the tips are unlocked.

To unlock tips, press ***h*** to be met with the following view under the **`Output`** tab:

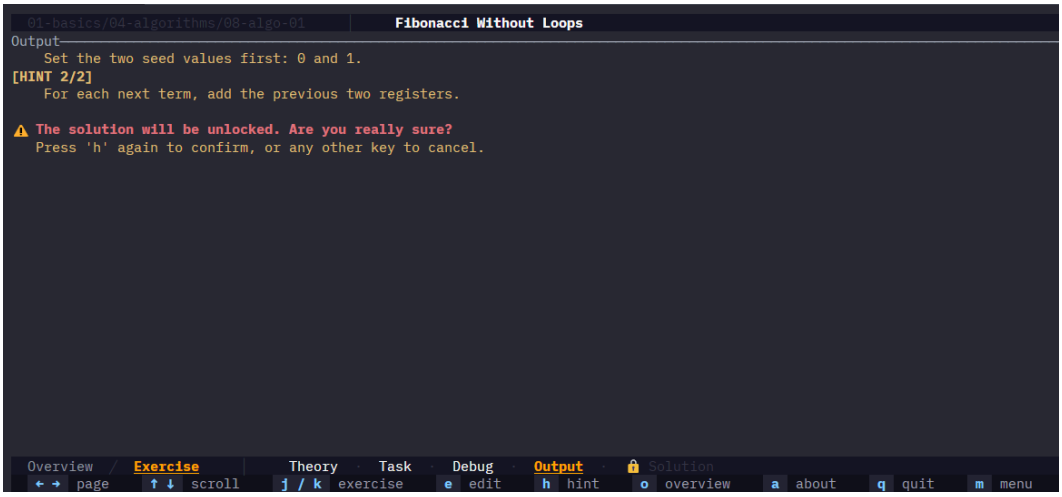


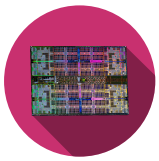
Figure 14 - Output tab

Once all tips unlocked, pressing **"h"** again unlocks the solution which shows the following view:

Figure 15 - Solution tab

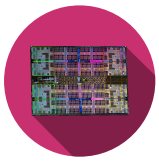
3.3.6 Shortcuts summary

- ← →: navigate through the different views of the exercise
- ↓ ↑: scroll in the views
- ***j***** / ***k***: go to previous/next exercise
- ***e***: open the exercise in the default editor
- ***h***: show tips + unlock solution
- ***o***: go back to the overview page - Section 3.2
- ***a***: show the about page
- ***q***** / ***Esc***: quit the program
- ***m***: show/hide the menu with the shortcuts for the current view



3.4 Troubleshooting

Problem	Source	Resolution steps
`rustup`, **`cargo`**, or **`rustc`** unavailable	Rust toolchain is not installed correctly	1. Re-run Rust installation from **`https://rust-lang.org/tools/install/`**. 2. Restart terminal session. 3. Verify with **`rustup --version`**, **`rustc --version`**, and **`cargo --version`**.
`lq` command not found	LangQuest is not installed or Cargo binaries are missing from **`PATH`**	1. Run **`cargo install --git https://github.com/tschinz/langquest`**. 2. Check **`cargo --version`** and **`lq --version`**. 3. Add Cargo bin directory to **`PATH`** and reopen terminal.
Program is ran but immediately exits	Wrong working directory or invalid exercise tree layout	1. Check terminal output. Most of the time, it says that no exercises were found. 2. Go to the root folder containing the exercise modules. 3. Confirm the folder structure matches the expected **`NN-module/NN-exercise`** pattern. 4. Start **`lq`** again from that root. 5. Raise an issue under https://github.com/tschinz/langquest/issues.
Pressing **`e`** does not open files in expected editor	File type associations are missing or point to another application	1. Reconfigure default apps for **`.rs`**, **`.py`**, **`.go`**, **`.cpp`**, **`.md`**, etc. 2. Close and reopen editor. 3. Relaunch **`lq`**.
Solution remains locked	Completion threshold not reached or tips not fully unlocked	1. Fix your code to pass the tests until threshold is reached. 2. Press **`h`** repeatedly to unlock all tips. 3. Press **`h`** again to unlock the solution tab.
Interface freezes or becomes unresponsive	Currently, **`lq`** works on a single thread. When an exercise is opened or the exercise file is saved, a compilation + execution of that file is performed. If either of these steps takes time, the interface may appear frozen.	1. Check that the exercise code does not contain an infinite loop. 2. Try compiling outside of **`lq`**. 3. Check if a newer version of LangQuest is available.



Glossary

***RISC-V* – RISC-V Instruction Set Architecture**: An open-source instruction set architecture for computer processors. 3, 3

***MIT* – MIT License**: A permissive free software license originating at the Massachusetts Institute of Technology (MIT). 3

***AI* – Artificial Intelligence**: Field of computer science focused on creating systems capable of performing tasks that typically require human intelligence. 9

***lq* – LangQuest**: An interactive programming exercise runner that supports multiple languages and provides real-time feedback. 3

***MSVC* – Microsoft Visual C++**: A compiler and development environment for C and C++ programming languages from Microsoft. 5, 5